

操作系统第二次实验报告

李汶峰

202300460158

2023 级网安一班

2025 年 3 月 24 日

1 实验目的

- 为 C/OS-II 加入时间片轮转调度器
- 每 8 个优先级分为一类，前 64 个优先级共分为 8 类，类内实施时间片轮转调度，类间实施优先级调度。

2 实验过程

2.1 任务一

Round Robin 是一种时间片轮转的 CPU 调度算法，主要用于多任务操作系统。其核心思想是将 CPU 时间划分为一个个固定长度的时间片，然后按顺序分配给各个就绪态的进程。每个进程在分配到的时间片内运行，直到时间片结束，如果进程在时间片结束时仍未完成，它会被放回就绪队列的末尾，等待下一次调度。CPU 不断循环调度队列中的进程，确保每个进程都能获得公平的 CPU 时间。

代码中使用了一个双向链表来实现就绪队列。每个任务 (Task) 由一个 OS_TCB 结构表示。相关函数的说明为：

- OSRdyTCBQueueFront：指向就绪队列的队首 (Front)
- OSRdyTCBQueueRear：指向就绪队列的队尾 (Rear)
- OSRdyTCBQueueNum：记录就绪队列中任务的数量

1. 在 os_core.c 开始处实现就绪任务队列：

OS_InitRdyQueue 函数用于初始化就绪队列，初始化就绪队列，将队首、队尾指针置为空，并将任务数量置为 0，代码如下：

```
1 void OS_InitRdyQueue ()
2 {
3     OSRdyTCBQueueFront = (OS_TCB *)0;
4     OSRdyTCBQueueRear  = (OS_TCB *)0;
5     OSRdyTCBQueueNum   = 0;
6 }
```

OSRdyQueueIn 函数用于将任务加入就绪队列，将该任务的时间片设置为最大值，并将该任务的尾指针指向 NULL，头指针指向当前队列的对尾。如果当前队列为空则将任务设置为队首。同时，将当前队尾任务的尾指针指向新任务，并更新就绪队列的尾指针为新任务。代码如下：

```
1 void OSRdyQueueIn(OS_TCB *ptcb)
2 {
3     ptcb->quantum = OS_SCHED_QUANTUM_MAX;
4     ptcb->OSRdyTCBNext = (OS_TCB *)0;
5     ptcb->OSRdyTCBPrev = OSRdyTCBQueueRear;
6     if (OSRdyTCBQueueFront == (OS_TCB *)0)
7         OSRdyTCBQueueFront = ptcb;
8     else
9         OSRdyTCBQueueRear->OSRdyTCBNext = ptcb;
10    OSRdyTCBQueueRear = ptcb;
11 }
```

OSRdyQueueOut 函数用于从就绪队列中取出任务，首先获取队首任务，如果队列为空则返回 NULL。然后更新队首指针为该任务指向的下一个任务，同样更新就绪队列的队尾指针和新任务的指针。代码如下：

```
1 OS_TCB* OSRdyQueueOut(void)
2 {
3     OS_TCB *ptcb = OSRdyTCBQueueFront;
4     if (ptcb == (OS_TCB *)0)
5         return (OS_TCB *)0;
6
7     OSRdyTCBQueueFront = ptcb->OSRdyTCBNext;
8     if (OSRdyTCBQueueFront == (OS_TCB *)0)
9         OSRdyTCBQueueRear = (OS_TCB *)0;
10    else
11        OSRdyTCBQueueFront->OSRdyTCBPrev = (OS_TCB *)0;
12    return ptcb;
13 }
```

2. 修改 OS_SchedNew 函数以实现 round robin 调度

当使能 round robin 调度标记大于 0 时，此时进入 round robin 调度，首先检查当前是否有任务在运行，或者当前任务的时间片是否耗尽，如果满足上述条件则进行任务切换。在切换时，如果当前任务有任务在运行则将其放回就绪队列中。然后从就绪队列的首部出列一个任务，更新全局变量 OSPrioHighRdy 为新任务的优先级，以进行当前任务的实行。修改部分的代码如下：

```
1 #if OS_SCHED_ROUND_ROBIN_EN > 0
2     if (OSTCBCur == 0 || OSTCBCur->quantum == 0)
3     {
4         if (OSTCBCur != 0)
5             OSRdyQueueIn(OSTCBCur);
6         OS_TCB* new_task = OSRdyQueueOut();
```

```

7         if (new_task != (OS_TCB *)0)
8             OSPrioHighRdy = new_task->OSTCBPrio;
9     }
10
11 #endif

```

3. 修改 OS_TCBInit 函数完成新任务的 TCB 入队就绪队列

这部分函数用于将一个任务加入到就绪队列中。代码的核心逻辑是维护一个双向链表结构的就绪队列，确保任务能够按顺序被调度。

```

1 #if OS_SCHED_ROUND_ROBIN_EN > 0
2     ptcb->quantum = OS_SCHED_QUANTUM_MAX;
3     if (OSRdyTCBQueueFront == (OS_TCB *)0) {
4         OSRdyTCBQueueFront = ptcb;
5         OSRdyTCBQueueRear = ptcb;
6     } else {
7         OSRdyTCBQueueRear->OSRdyTCBNext = ptcb;
8         ptcb->OSRdyTCBPrev = OSRdyTCBQueueRear;
9         OSRdyTCBQueueRear = ptcb;
10    }
11    ptcb->OSRdyTCBNext = (OS_TCB *)0;
12 #endif

```

4. 修改 OSTimeTick 函数完成减少当前任务剩余时间的操作、和从阻塞态转到就绪态的任务就绪队列的操作。

在 OSTimeTick 中的这段代码的功能是减少当前任务的时间片。每次调度时，时间片会随着时间逐渐减少，直到时间片耗尽（quantum 为 0），此时任务需要被切换到就绪队列的末尾，等待下一次调度。

```

1 #if OS_SCHED_ROUND_ROBIN_EN > 0
2     OSTCBCur->quantum--;
3 #endif

```

任务一实验结果：可以看到 task1 和 task2 都实现了 round robin 调度，依次运行。

```
Hello from Task 2!  
Hello from Task 2!  
Hello from Task 2!  
Hello from Task 2!  
Hello from Task 2!  
Hello from Task 2!  
Hello from Task 2!  
Hello from Task 2!  
Hello from Task 2!  
Hello from Task 1!  
Hello from Task 1!  
Hello from Task 1!  
Hello from Task 1!  
Hello from Task 1!  
Hello from Task 1!  
Hello from Task 1!  
Hello from Task 1!  
Hello from Task 1!  
Hello from Task 1!
```

图 1: 任务一实验结果

3 实验感悟

在本次实验中，我通过为 $\mu\text{C}/\text{OS-II}$ 添加时间片轮转调度器，深入理解了实时操作系统的任务调度机制。 $\mu\text{C}/\text{OS-II}$ 原本采用严格的优先级抢占调度，高优先级任务始终优先执行。而时间片轮转调度要求同一优先级的任务共享 CPU 时间。OS_SchedNew：原函数通过位图查找最高优先级任务，现需修改为从就绪队列中轮转选择任务。OS_TCBInit：新任务创建时，需根据其优先级判断是否加入轮转队列，避免干扰高优先级任务的抢占逻辑。SysTick 中断处理：在时间片递减逻辑中，需在时钟中断服务例程中更新剩余时间片，并在时间片归零时触发调度。这让我对中断与任务调度的交互有了更直观的认识，时间片轮调度的实现需要兼顾数据结构、调度策略和系统稳定性。本次实验不仅锻炼了代码实践能力，更让我认识到操作系统内核设计的精妙之处。